

Walchand College of Engineering, Sangli

7CS353 Machine Learning Lab — Assignment 1

Python Fundamentals Through Your First Dataset Class

Name - Atharv Vinayak Kulkarni

PRN - 245109074

Class - TY CSE(B)

Solutions for All Tasks from Assignment 1

A1. Variables and Printing

Task: Create variables for your own name and expected marks, and print a similar sentence. Explore output formatting in Python.

Solution:

```
[4]: myname="atharv"  
     mymarks=99.99  
     print(myname,"scored",mymarks,"in Test A")  
atharv scored 99.99 in Test A
```

A2. Lists and Indexing

Task: Print the name and Test 1 marks of the 5th student in one line.

Solution:

```
[6]: print(names[4],test1[4])  
Kabir 62
```

A3. Loops

Task: Modify the loop to print Test 2 marks and the improvement (Test2 – Test1) for each student. Explore enumerate().

Solution:

```
[8]: for i,name in enumerate(names):
      print(f'name: {name},test1: {test1[i]},test2: {test2[i]},test1-test2: {test2[i]-test1[i]}')

name: Aarav,test1: 56,test2: 61,test1-test2: 5
name: Diya,test1: 78,test2: 82,test1-test2: 4
name: Ishaan,test1: 45,test2: 50,test1-test2: 5
name: Meera,test1: 88,test2: 90,test1-test2: 2
name: Kabir,test1: 62,test2: 58,test1-test2: -4
name: Anaya,test1: 91,test2: 94,test1-test2: 3
name: Rohan,test1: 40,test2: 38,test1-test2: -2
name: Sara,test1: 73,test2: 70,test1-test2: -3
name: Vivaan,test1: 67,test2: 72,test1-test2: 5
name: Priya,test1: 85,test2: 88,test1-test2: 3
name: Arjun,test1: 52,test2: 47,test1-test2: -5
name: Nisha,test1: 95,test2: 97,test1-test2: 2
```

A4. Conditionals

Task: Print only the names of students scoring more than 80 in Test 1 and count how many such students there are.

Solution:

```
[9]: count=0
      for i in range(len(names)):
          if test1[i]>80:
              print(names[i])
              count+=1
      print("count of students scoring above 80 is:",count)

Meera
Anaya
Priya
Nisha
count of students scoring above 80 is: 4
```

A5. Functions

Task: Complete average(values) to return the average of any list. Use it for Test1, Test2 and Attendance.

Solution:

```
[10]: def average(values):
        total=0
        for v in values:
            total+=v
        return total/len(values)

print(average(test1))
print(average(test2))
print(average(attendance))

69.33333333333333
70.58333333333333
79.33333333333333
```

A5 Checkpoint

Task: Write whether Test 2 average being higher than Test 1 indicates overall class improvement. Can average alone prove every student improved?

Solution:

a) $\text{score}(\text{test1}) < \text{score}(\text{test2})$ because students performed good in test2 as compared to test1
b) for alone performance we need to compare the score of each student from test1 and test2 from there we will get alone performance

B1. NumPy Vectors

Task: Create NumPy arrays and compute improvement using vector subtraction.

Solution:

```
[25]: import numpy as np
      t1=np.array(test1)
      t2=np.array(test2)
      att=np.array(attendance)
      print(t2-t1)

      [ 5  4  5  2 -4  3 -2 -3  5  3 -5  2]
```

B2. Summary Statistics

Task: Compute mean, variance and standard deviation of Test 1. Verify mean with average() function.

Solution:

```
[12]: print(np.mean(t1))
      print(np.var(t1))
      print(np.std(t1))
      print(np.mean(t1)==average(test1))

      69.33333333333333
      316.7222222222223
      17.796691327946952
      True
```

B3. Interpreting Spread

Task: Compare standard deviation of Test 1 and Attendance. State which is more spread out and what large standard deviation means.

Solution:

```
[13]: print(np.std(t1))
      print(np.std(att))
      17.796691327946952
      14.522013940527977
```

B4. Geometry

Task: Find the student closest to Nisha in 3D space using Euclidean distance.

Solution:

```
[14]: meera=np.array([88,90,95])
      kabir=np.array([62,58,70])
      dist=np.sqrt(np.sum((meera-kabir)**2))
      print(dist)
      48.218253804964775

[15]: nisha_index=names.index("Nisha")
      nisha=np.array([test1[nisha_index],test2[nisha_index],attendance[nisha_index]])
      min_dist=float("inf")
      closest=""
      for i in range(len(names)):
          if names[i]=="Nisha":
              continue
          student=np.array([test1[i],test2[i],attendance[i]])
          d=np.sqrt(np.sum((nisha-student)**2))
          if d<min_dist:
              min_dist=d
              closest=names[i]
      print(closest,min_dist)
      Anaya 5.0990195135927845
```

C1. Load and Inspect

Task: Load marks.csv. Use head(), shape and describe(). Explain count, mean, std, min, 25%, 50%, 75%, max.

Solution:

```
[17]: df=pd.read_csv("marks.csv")
print(df.head())
print(df.shape)
print(df.describe())
```

	name	test1	test2	attendance
0	Aarav	56	61	72
1	Diya	78	82	88
2	Ishaan	45	50	65
3	Meera	88	90	95
4	Kabir	62	58	70

(12, 4)

	test1	test2	attendance
count	12.000000	12.000000	12.000000
mean	69.333333	70.583333	79.333333
std	18.588038	19.888134	15.167749
min	40.000000	38.000000	55.000000
25%	55.000000	56.000000	68.750000
50%	70.000000	71.000000	79.000000
75%	85.750000	88.500000	92.750000
max	95.000000	97.000000	99.000000

C2. Selecting and Filtering

Task: Print only name and test1 columns. Print rows with attendance > 85.

Solution:

```
[18]: print(df[["name", "test1"]])
```

	name	test1
0	Aarav	56
1	Diya	78
2	Ishaan	45
3	Meera	88
4	Kabir	62
5	Anaya	91
6	Rohan	40
7	Sara	73
8	Vivaan	67
9	Priya	85
10	Arjun	52
11	Nisha	95

```
[19]: print(df[df["attendance"]>85])
```

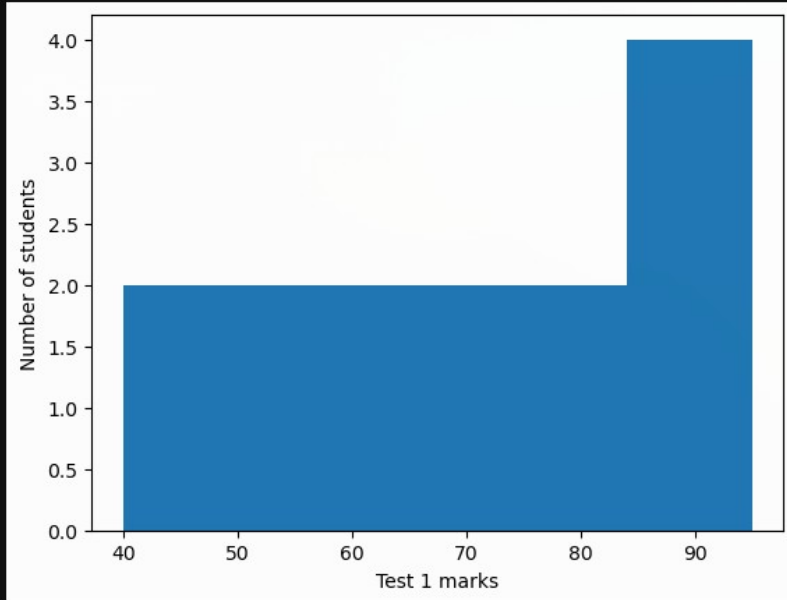
	name	test1	test2	attendance
1	Diya	78	82	88
3	Meera	88	90	95
5	Anaya	91	94	98
9	Priya	85	88	92
11	Nisha	95	97	99

C3. Histogram

Task: Plot histogram of Test 1 marks. Comment on the distribution shape.

Solution:

```
[20]: import matplotlib.pyplot as plt
plt.hist(df["test1"],bins=5)
plt.xlabel("Test 1 marks")
plt.ylabel("Number of students")
plt.show()
```



Observations:

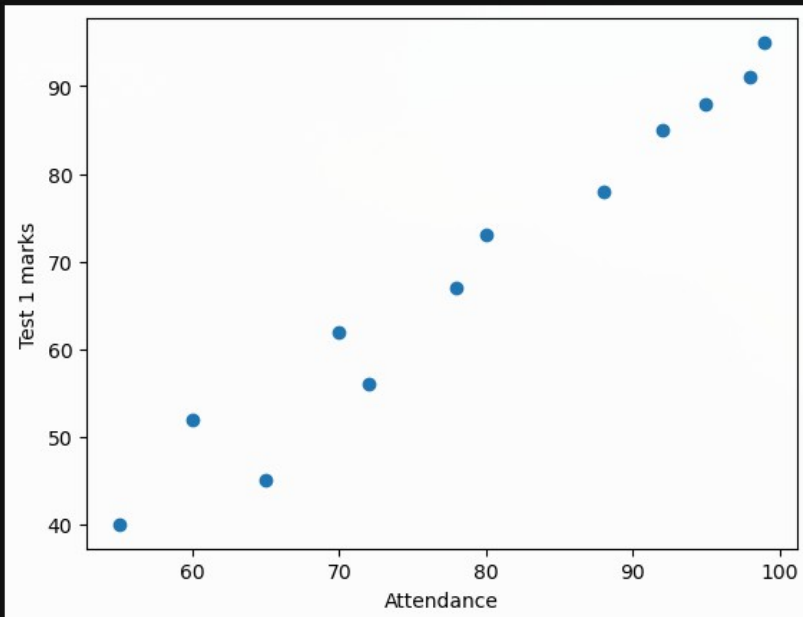
The histogram I got is not roughly bell-shaped I can see histogram is concentrated toward the higher marks (around 85–95), while only a few students have lower scores. So I can say this is right skewed histogram.

C4. Scatter Plot

Task: Plot attendance vs Test 1 marks. State whether higher attendance tends to relate to higher marks and whether a straight line fits.

Solution:

```
[21]: plt.scatter(df["attendance"],df["test1"])  
plt.xlabel("Attendance")  
plt.ylabel("Test 1 marks")  
plt.show()
```



Answer:

Yes, students with higher attendance got to score more so i see that the points trend upward from left to right.

Yes, from the above graph observation i can see that the straight line can be drawn with positive slope from origin

D1. Grade Function

Task: Create grade() function, add grade column for Test 2, print grade counts.

Solution:

D1. Grading Function

```
[22]: def grade(marks):  
      if marks >= 85:  
          return "A"  
      elif marks >= 70:  
          return "B"  
      elif marks >= 50:  
          return "C"  
      else:  
          return "F"  
  
      df["grade"] = df["test2"].apply(grade)  
      print(df["grade"].value_counts())
```

```
grade  
A    4  
C    3  
B    3  
F    2  
Name: count, dtype: int64
```

D2. Outlier Effect

Task: Change Rohan's Test 1 marks to 400. Recompute mean and standard deviation. Explain effect of outlier.

Solution:

```
[24]: df.loc[df["name"]=="Rohan", "test1"] = 400  
      print(df["test1"].mean())  
      print(df["test1"].std())
```

```
99.33333333333333  
96.04954529572304
```

Answer:

so changing Rohan's test1 value to 400 makes the mean sharply upward since it's now far larger than every other score.

the standard deviation increases very high, since one value is much bigger than others.

this shows how a single outlier or data-entry error can change summary statistics for an entire dataset.

D3. Optional Stretch

Task: Compute variance manually using a loop without `np.var` and verify the result.

Solution:

```
[23]: mean_t1=sum(test1)/len(test1)
      total=0
      for v in test1:
          total+=(v-mean_t1)**2
      variance=total/len(test1)
      print(variance)
      print(np.var(t1))
      print(round(variance,6)==round(np.var(t1),6))

316.7222222222222
316.7222222222223
True
```